Name:_____

ECE 223: Computer Systems Engineering **Fall 2014**

Exam 2 **Monday, November 10**

1. For each description (a)-(j) specify the matching term from the list of terms. 10 points

1. AVL tree
2. Binary tree
3. Coalescing
4. Compacting
5. Complete
6. Complexity class
7. Dominant term
8. Equilibrium
9. Extended
10. FIFO
11. Fragmentation
12. Frame pointer
13. Free list
14. Garbage collection
15. Heap
16. Height
17. In-order
18. Internal path length
19. LIFO
20. Leaf
21. Level
22. Level-order
23. Pre-order
24. Post-order
25. Queue
26. Recurrence relation
27. Root
28. Roving pointer
29. Stack
30. Trie
31. Two-three tree

a) ____: An ADT useful for processing nested structures.

b) ____: When describing the O-notation of an algorithm we focus on this.

c) ____: Either the empty tree or a node that has left and right subtrees that are binary trees.

d) ____: In a tree, the sum of the lengths of the paths from the root to all nodes.

e) ____: A traversal order in which the nodes in a binary search tree are visited in sorted order.

f) ____: A method to counteract the tendency toward fragmentation.

g) ____: A zone of memory organized to support dynamic memory allocation of blocks of storage of differing sizes.

h) ____: A tree that is insensitive to the order of insertion of the keys, but is sensitive to clustering in the spelling of the prefixes of the keys.

i) ____: A binary tree that has its empty binary subtrees explicitly represented

j) ____: A characterization of running times for different algorithms that share the same family of curves.

2. Short answer questions (16 points)

2.1. What is a benefit of a best-fit search policy compared to a first-fit search policy for a heap?

2.2. True or False: An $O(n)$ algorithm will always finish before an $O(n^2)$ algorithm. If the statement is false, describe why.

2.3. For an extended binary tree with n nodes, if the internal path length is I, then the external path length, E, equals:

2.4. For a complete binary tree with *n* nodes the number of levels in the tree equals:

2.5. What is the big-O notation for organizing the items in an initially unorganized complete binary tree into a heap?

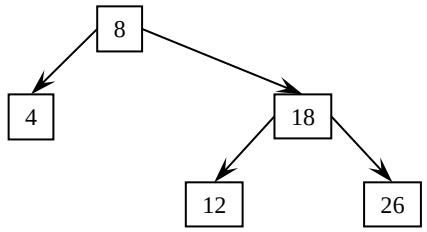
2.6. Give the equation that relates Count, Front, and Rear for a circular queue representation utilizing an array with size *N*. (Insert at Rear, and remove at Front.)

2.7. For a complete binary tree with *n* nodes using an array implementation, if a node is located in array index *i*, give the array index of its right child and the condition on *i* such that it has a right child.

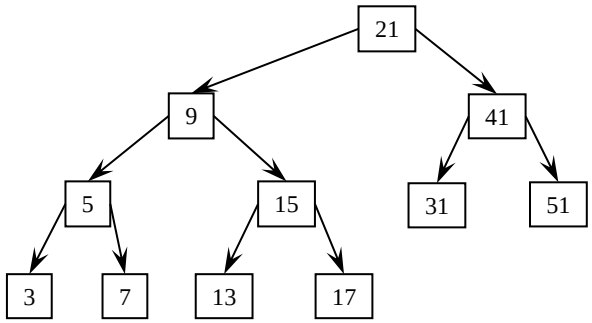
2.8. True or False: An inorder traversal always visits the elements of a binary search tree in the same order, regardless of the order in which the elements were inserted.

3. Trees. (30 points)

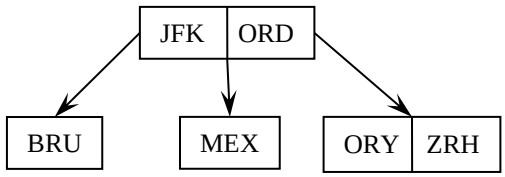
3.1. Insert 30 into the following AVL tree. You need to draw the final tree.



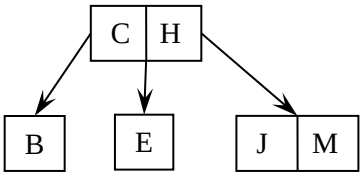
3.2. Insert 11 into the following AVL tree. You need to draw the final tree.



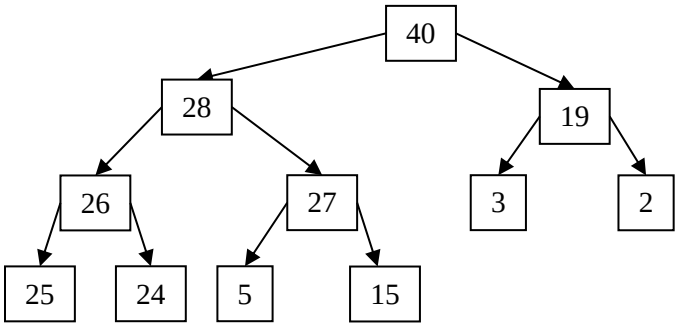
3.3. Insert GCM and then NRT into the following Two-three tree. You need to draw the final tree.



3.4. Insert L into the following Two-three tree. You need to draw the final tree.



3.5. A priority queue is implemented as a heap, and has the initial state shown below. Insert 29 into the priority queue. Draw the final state of the heap



3.6. The following priority queue is implemented as a heap using a sequential representation. Show the sequential representation of the heap after the high priority item is removed from the queue.

46	43	37	13	10	1	32	5	8	4

4. Algorithm design. (24 points)

4.1. Consider a sorted list. For an implementation using an array, the binary search algorithm has a lower complexity class than the sequential search algorithm. For our two-way linked list module in MP2, using a sorted linked list we cannot implement the binary search algorithm that has the same complexity class as for an implementation with an array. What is the primary problem with implementing a binary search algorithm for a two-way linked list that is not a problem with an array?

4.2. Determine the runtime complexity class to execute the algorithm **method**. Assume **the code** executes in an amount of time that does not depend on **n** (i.e., the execution time for **the code** can be modeled as a constant c).

```
void method(int n) {
      int i, j;
      for( j = 1; j <= n ; j++) {
            i = j;
            while (i > 0) {
                  /* the code */
                  i = i/2;
            }
      }
}
```

For questions 4.3 and 4.4: Consider an application in which a large number of Internet packets must be stored and analyzed. The application alternates between two phases. During the first phase, a large number of packets are collected and stored, and let the number of packets collected be *n*. During the second phase the packets are analyzed one-by-one by removing them from the storage container. When the storage container is empty, the application returns to the first phase and the process repeats. Each packet has an IP address (assume the address is a positive 32-bit integer), and the packets must be analyzed in an increasing order based on their IP addresses. Assume that when the packets are collected there is no particular order of the IP address (i.e., the packets appear to have random IP addresses) but some packets may have the same IP address. You are to design the data structure and algorithm for the storage container.

4.3. Consider your linked-list implementation from MP2. During phase one, packets are inserting using your `list_insert_sorted(packet_list, packet_ptr)` function and during phase two, your `packet_ptr = list_remove(packet_list, LISTPOS_HEAD)` command is used to remove the packets in order. Considering that there are *n* packets that must be processed, what is the computational complexity in O-notation for phase one (using `list_insert_sorted)`? For phase two (using `list_remove)`? For both answers explain the reason for your answer.

4.4. Describe an alternative design for an implementation that significantly reduces the overall time for both phases (i.e., inserting *n* packets, and then removing them in order). What is the computational complexity in O-notation for your new `insert()`? For your new `remove()`? For both answers explain the reason for your answer. Clearly explain why this new design yields a substantial reduction in the overall time.

5. Recursion. (20 points)

Write a recursive function to find the value of the key in a binary search tree that is the median. For simplicity, assume the tree contains an odd number of keys and the number of keys in the tree is known when the function is called. You may use auxiliary functions if needed.

There is a pointer to the header block of the tree of type `binary_tree_t`, and it contains a pointer to the root of the tree called `root` and a member called `count` which is the number of keys contained in the tree. The items of the tree have type `tree_node_t`, and have `left` and `right` pointers and an integer key value called `key`. For the functions you are asked to develop, you cannot modify the tree or store any information in the nodes of the tree or the tree header. Other than the `left` and `right` pointers, your function cannot access any other fields in a `tree_node_t`.

```
int tree_median(binary_tree_t *T)
```